

Python: exercises on branching

This notebook was generated in **student** mode.

Exercise 1: Positive, Negative, or Zero

Exercise Description

Write a program that determines whether a given number is positive, negative, or zero.

Use the number `num = -5` and store the result in variable `result` ("positive", "negative", or "zero").

Your solution

```
num = -5
# Write your solution here
result =
```

Test your solution

```
assert result == "negative"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Even or Odd Number

Exercise Description

Write a program that determines whether a given integer is even or odd.

Use the number `num = 7` and store the result in variable `result` ("even" or "odd").

Your solution

```
num = 7
# Write your solution here
result =
```

Test your solution

```
assert result == "odd"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Grade Evaluator

Exercise Description

Write a program that converts a numerical score (0-100) to a letter grade:

- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- 0-59: F

Use the score `score = 85` and store the letter grade in variable `result`.

Your solution

```
score = 85  
# Write your solution here  
result =
```

Test your solution

```
assert result == "B"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Divisibility Checker

Exercise Description

Write a program that checks if a given number is divisible by both 3 and 5.

Use the number `num = 30` and store the result in variable `result` ("divisible by both" or "not divisible by both").

Your solution

```
num = 30
# Write your solution here
result =
```

Test your solution

```
assert result == "divisible by both"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Leap Year Checker

Exercise Description

Write a program that determines whether a given year is a leap year or not.

A leap year is:

- Divisible by 4 but not by 100, OR
- Divisible by 400

Use the year `year = 2024` and store the result in variable `result` ("leap year" or "not leap year").

Your solution

```
year = 2024
# Write your solution here
result =
```

Test your solution

```
assert result == "leap year"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Vowel or Consonant

Exercise Description

Write a program that determines whether a given letter is a vowel or consonant.

Use the letter `letter = "A"` and store the result in variable `result` ("vowel" or "consonant").

Your solution

```
letter = "A"  
# Write your solution here  
result =
```

Test your solution

```
assert result == "vowel"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Password Validator

Exercise Description

Write a program that validates a password. If the password is "password123", grant access, otherwise deny access.

Use the password `password = "password123"` and store the result in variable `result` ("Access granted" or "Access denied").

Your solution

```
password = "password123"  
# Write your solution here  
result =
```

Test your solution

```
assert result == "Access granted"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Largest of Three Numbers

Exercise Description

Write a program that finds the largest of three given numbers.

Use the numbers `num1 = 15`, `num2 = 8`, and `num3 = 22`. Store the largest number in variable `result`.

Your solution

```
num1 = 15
num2 = 8
num3 = 22
# Write your solution here
result =
```

Test your solution

```
assert result == 22
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Day of the Week

Exercise Description

Write a program that converts a number (1-7) to the corresponding day of the week:

- 1: Monday
- 2: Tuesday
- 3: Wednesday
- 4: Thursday
- 5: Friday
- 6: Saturday
- 7: Sunday
- Any other number: "Invalid number"

Use the number `day = 5` and store the day name in variable `result`.

Your solution

```
day = 5
# Write your solution here
result =
```

Test your solution

```
assert result == "Friday"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Simple Calculator

Exercise Description

Write a program that performs basic arithmetic operations (+, -, *, /) on two numbers based on the given operation.

Use `num1 = 10.0`, `num2 = 3.0`, and `operation = "+"`. Store the result in variable `result`. Handle division by zero with an error message.

Your solution

```
num1 = 10.0
num2 = 3.0
operation = "+"
# Write your solution here
result =
```

Test your solution

```
assert result == 13.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Triangle Type Classifier

Exercise Description

Write a program that determines the type of triangle based on the lengths of its three sides:

- Equilateral: All sides are equal
- Isosceles: Two sides are equal
- Scalene: No sides are equal

Use `side1 = 5.0`, `side2 = 5.0`, and `side3 = 8.0`. Store the triangle type in variable `result`.

Your solution

```
side1 = 5.0
side2 = 5.0
side3 = 8.0
```

```
# Write your solution here  
result =
```

Test your solution

```
assert result == "isosceles"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: BMI Calculator with Categories

Exercise Description

Calculate the Body Mass Index (BMI) and categorize it according to standard health categories:

- Underweight: BMI < 18.5
- Normal weight: $18.5 \leq \text{BMI} < 25$
- Overweight: $25 \leq \text{BMI} < 30$
- Obese: BMI ≥ 30

BMI formula: weight (kg) / (height (m))²

Use `weight = 70` kg and `height = 1.75` m. Store the BMI category in variable `result`.

Your solution

```
weight = 70 # kg
height = 1.75 # m
# Write your solution here
result =
```

Test your solution

```
assert result == "Normal weight"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Rock Paper Scissors Game

Exercise Description

Implement the logic for a Rock Paper Scissors game. Determine the winner based on the classic rules:

- Rock beats Scissors
- Scissors beats Paper
- Paper beats Rock
- Same choice results in a tie

Use `player1 = "rock"` and `player2 = "scissors"`. Store the result in variable `result` ("Player 1 wins", "Player 2 wins", or "It's a tie").

Your solution

```
player1 = "rock"  
player2 = "scissors"  
# Write your solution here  
result =
```

Test your solution

```
assert result == "Player 1 wins"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Temperature Converter

Exercise Description

Create a temperature converter that converts between Celsius and Fahrenheit based on the conversion type:

- "C to F": Convert Celsius to Fahrenheit using $F = (C \times 9/5) + 32$
- "F to C": Convert Fahrenheit to Celsius using $C = (F - 32) \times 5/9$

Use `temperature = 25.0` and `conversion_type = "C to F"`. Store the converted temperature in variable `result`.

Your solution

```
temperature = 25.0
conversion_type = "C to F"
# Write your solution here
result =
```

Test your solution

```
assert result == 77.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Movie Ticket Price Calculator

Exercise Description

Calculate movie ticket price based on age and day of the week:

- Regular price: \$12
- Child (under 12): \$8
- Senior (65 and over): \$10
- Tuesday discount: 20% off all tickets
- Weekend surcharge (Saturday/Sunday): \$2 extra

Use `age = 25` and `day = "Tuesday"`. Store the ticket price in variable `result`.

Your solution

```
age = 25
day = "Tuesday"
# Write your solution here
result =
```

Test your solution

```
assert result == 9.6 # $12 * 0.8 = $9.60
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: Traffic Light Simulator

Exercise Description

Simulate a traffic light system that determines the action based on the current light color:

- "red": "Stop"
- "yellow": "Prepare to stop"
- "green": "Go"
- Any other color: "Invalid light color"

Use `light_color = "green"` and store the action in variable `result`.

Your solution

```
light_color = "green"  
# Write your solution here  
result =
```

Test your solution

```
assert result == "Go"
```

Debug your solution

